



Why Database ?

Program Variables??

RAM - Volatile

Files? CSV?

Multiple users?

Efficient Sorting , Searching?

Databases are :

Optimized

Scalable

Concurrent

Types of Databases

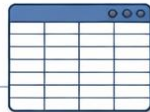
SQL DATABASE

Tables

Relational Table

id	name	age	email
1	John	30	john@email.com
2	Sarah	25	sarah@email.com
3	Mike	35	mike@email.com

- Rows & Columns
- Fixed Schema



Structured Table

VS

NoSQL DATABASE

JSON Documents

JSON Document

```
{  
  'name': "John",  
  'age': 30,  
  "email": "john@email.com"  
}
```

- Key–Value Pairs
- Flexible Schema



Python Dictionary



MongoDB

Database

Collection - Users

```
{  
  username: "aryansharma05",  
  Gender: "male"  
}
```

```
{  
  username: "sigmaRohan",  
  Gender: "male"  
}
```

Collection - Likes

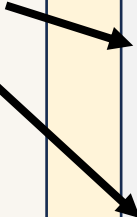
```
{  
  username: "aryansharma05",  
  postID: "4f8wefh3"  
}
```

```
{  
  username: "betaAkash",  
  postID: "nb22mo09"  
}
```

Collection - ...

```
{  
  ... : "...",  
  ... : "..."  
}
```

**Document
s**





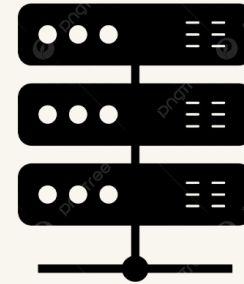
MongoDB

Community Server



Runs on your local machine

MongoDB Atlas



Runs on hosted server

Provided by MongoDB – Free Tier Available



MongoDB – Connection in Express

```
const mongoURI= "XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"

mongoose.connect(mongoURI, {
  tls: true,
  tlsAllowInvalidCertificates: true,
}).then(() => {
  console.log('Connected to MongoDB');
}).catch((error) => {
  console.error('MongoDB connection error:', error.message);
});
```



MongoDB – Defining Schema

```
// Define Schema
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true
    },
    email: {
      type: String,
      required: true,
      unique: true
    },
    rollnumber: {
      type: String,
      required: true,
      unique: true
    },
    phone: {
      type: String
    }
  },
  {
    timestamps: true
  }
)
```



MongoDB – Adding Documents

```
const AddUserFunction= async () => {  
  const newUser = new User({  
    name: 'John Doe',  
    email: 'sample@zense.com',  
    rollnumber: '123456',  
    phone: '123-456-7890'  
  });  
  try {  
    const savedUser = await newUser.save();  
    console.log('User saved successfully:', savedUser);  
  } catch (error) {  
    console.error('Error saving user:', error.message);  
  }  
}
```

```
_id: ObjectId('695e62611cb2c6793230a654')  
name : "John Doe"  
email : "sample@zense.com"  
rollnumber : "123456"  
phone : "123-456-7890"  
createdAt : 2026-01-07T13:40:49.821+00:00  
updatedAt : 2026-01-07T13:40:49.821+00:00  
__v : 0
```



MongoDB – Searching Documents

```
const FindUserByEmail= async () => {  
  try {  
    const user = await User.findOne({ email: "sample@zense.com" });  
    if (user) {  
      console.log('User found:', user);  
    } else {  
      console.log('No user found with that email.');    }  
  } catch (error) {  
    console.error('Error finding user:', error.message);  
  }  
}
```




MongoDB – Searching Documents

```
const findUserbyRollNumber = async () => {  
  try {  
    const user = await User.findOne({ rollnumber: "123456" });  
    if (user) {  
      console.log('User found:', user);  
    } else {  
      console.log('No user found with that roll number.');    }  
  }  
  catch (error) {  
    console.error('Error finding user:', error.message);  
  }  
}
```

Thank You!